

Techniki Mikroprocesorowe

Sprawozdanie – Laboratorium 3

Autor: Kamil Sitarski

Numer albumu: [REDACTED]

Grupa dziekańska: [REDACTED]

Data wykonania ćwiczenia: 28.04.2026r.

Polecenie 1

1. Opis zadania

Celem zadania była implementacja programowej pętli opóźniającej „timer10ms()”, wykorzystującej sprzętowy Timer 0 w trybie CTC. W przeciwieństwie do funkcji `_delay_ms()`, tryb CTC pozwala na automatyczne zerowanie licznika po osiągnięciu zadanej wartości, co zwiększa precyzję i ułatwia cykliczne odmierzenie czasu.

2. Obliczenia

$$F_{CPU} = 1\,000\,000\text{ Hz}$$

$$t_{zadany} = 10\text{ ms} = 0,01\text{ s}$$

$$N = 256 \text{ (wartość preskalera)}$$

$$\begin{aligned} OCR0 &= \frac{F_{CPU} \cdot t_{zadany}}{N} - 1 \\ OCR0 &= \frac{1\,000\,000\text{ Hz} \cdot 0,01\text{ s}}{256} - 1 \\ OCR0 &= \frac{10\,000}{256} - 1 \\ OCR0 &= 39,0625 - 1 \approx 38 \end{aligned}$$

3. Opis kodu

Program realizuje sprzętowe odmierzenie czasu przy użyciu Timera 0 w trybie CTC. Poniżej przedstawiono analizę najważniejszych fragmentów implementacji:

- **Inicjalizacja rejestrów:** Na początku funkcji `main` następuje konfiguracja sprzętu:

```
OCR0 = 38; // Wartość porównania obliczona dla 10 ms
TCCR0 |= (1<<WGM01) | (1<<CS02); // Ustawiono na: tryb CTC, preskaler 256
```

Rejestr `TCCR0` (Timer/Counter Control Register) odpowiada za logikę działania układu, a `OCR0` wyznacza moment wyzerowania licznika.

- **Pętla oczekiwania:** Procedura `timer10ms()` monitoruje rejestr flag przerwań:

```
while(!(TIFR & (1<<OCF0))); // Oczekiwanie na flagę dopasowania
TIFR |= (1<<OCF0); // Ręczne zerowanie flagi poprzez wpisanie '1'
```

Zastosowanie `while` z negacją flagi `OCF0` zapewnia, że procesor wstrzyma działanie do momentu, aż licznik doliczy do zadanej wartości 38.

- **Skalowanie czasu:** Aby uzyskać pełną sekundę, wykorzystano funkcję pomocniczą z pętlą iteracyjną:

```
void czas_1s(){
    for(int i=0; i<100;i++){
        timer10ms();           // Wywołanie 100 razy (100 * 10 ms = 1 s)
    }
    PORTA ^= (1<<0);           // Negacja stanu pinu PA0 po upływie 1 s
}
```

Operacja `PORTA ^= (1<<0)` zmienia stan logiczny diody LED na przeciwny co każdą sekundę, co pozwala na wizualną weryfikację poprawności działania programu.

4. Kod:

```
/*
 * 28_04_2026_KS.c
 *
 * Created: 28.04.2026 16:37:29
 * Author : student_pl
 */
#define F_CPU 1000000UL
#include <avr/io.h>

void timer10ms(void){
    while(!(TIFR & (1<<OCF0)));
    TIFR |= (1<<OCF0);
}

void czas_1s(){
    for(int i=0; i<100;i++){
        timer10ms();
    }
    PORTA ^= (1<<0);
}

int main(void)
{
    OCR0 = 38;
    DDRA = 0b00000001;
    PORTA = 0x00;
    TCCR0 |= (1<<WGM01) | (1<<CS02);
    while (1)
    {
        czas_1s();
    }
}
```

Polecenie 2

1. Opis zadania

Zadanie polegało na konfiguracji Timera 0 do pracy w trybie NORMAL. Zamiast wewnętrznego zegara procesora, źródłem sygnału taktującego była linia zewnętrzna T0 (PB0), do której podłączono przycisk. Licznik został skonfigurowany w trybie NORMAL, co oznacza, że rejestr TCNT0 inkrementuje się z każdym wykrytym zboczem na pinie wejściowym. Przetestowano konfigurację dla zbocza narastającego oraz opadającego. Program pobiera aktualną wartość z rejestru TCNT0 i wystawia ją na Port A, co pozwala na obserwację liczby naciśnięć na diodach. Gdy flaga TOV0 w rejestrze TIFR jest podniesiona (licznik przekroczył wartość 255) program zatrzymuje pracę licznika (zeruje bity CS00 – CS02 w rejestrze TCCR0).

2. Opis Kodu

Program konfiguruje Timer 0 do pracy w trybie licznika zdarzeń zewnętrznych, zliczając impulsy generowane przez przycisk. Poniżej znajduje się analiza kluczowych fragmentów kodu:

- **Konfiguracja wejść/wyjść:**

```
DDRB &= ~(1<<0); // Ustawienie pinu PB0 jako wejście
PORTB |= (1<<0); // Włączenie rezystora podciągającego (pull-up) na pinie PB0
```

Dzięki włączeniu rezystora pull-up (podciągającego do zasilania), pin w stanie spoczynku ma stan wysoki, a naciśnięcie przycisku zwraca go do masy.

- **Uruchomienie licznika zewnętrznego:**

```
TCCR0 |= (1<<CS02) | (1<<CS01) | (1<<CS00); // Wybór źródła zegara: pin T0 (PB0),  
zbocze narastające.
```

Ustawienie wszystkich trzech bitów CS w rejestrze TCCR0 sprawia, że każda zmiana stanu na pinie T0 (PB0) inkrementuje rejestr TCNT0.

- **Odczyt i wyświetlanie wyniku:**

```
_delay_ms(200); // Oczekiwanie zapobiegające zbyt szybkiemu odświeżaniu danych
PORTA = TCNT0; // Wyświetlenie aktualnej liczby impulsów na Porcie A
```

Program cyklicznie pobiera wartość z rejestru TCNT0 i przesyła ją na diody LED, co umożliwia obserwację zliczonych impulsów. Zastosowane opóźnienie pomaga wyeliminować błędy wynikające z drgań styków przycisku.

- **Obsługa przepiętnienia:**

```
if(TIFR & (1<<TOV0)){ // Sprawdzenie flagi TOV0 w rejestrze TIFR
    TCCR0 &= ~(1<<CS02)|(1<<CS01)|(1<<CS00); // Odłączenie źródła zegara (zatrzymanie)
    TIFR |= (1<<TOV0); // Wyzerowanie flagi
}
```

Gdy rejestr TCNT0 przekroczy wartość 255, następuje detekcja przepiętnienia. Program reaguje na to zatrzymaniem pracy timera poprzez wyzerowanie bitów konfiguracji zegara w rejestrze TCCR0.

3. Kod

```
/*
 * 28_04_2026_KS.c
 *
 * Created: 28.04.2026 16:37:29
 * Author : student_pl
 */
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRA = 0xFF;
    DDRB &= ~(1<<0);
    PORTA = 0x00;
    PORTB |= (1<<0);
    TCCR0 |= (1<<CS02) | (1<<CS01) | (1<<CS00);
    TCNT0 = 250;
    while (1){
        _delay_ms(200);
        PORTA = TCNT0;
        if(TIFR & (1<<TOV0)){
            TCCR0 &= ~((1<<CS02)|(1<<CS01)|(1<<CS00));
            TIFR |= (1<<TOV0);
        }
    }
    return 0;
}
```

Polecenie 3

1. Opis zadania

Zadanie polegało na wygenerowaniu sygnału prostokątnego o okresie równym 200 ms i asymetrycznym wypełnieniu 75% . Wymagało to dynamicznej zmiany wartości rejestru porównania OCR0 w trakcie pracy programu.

2. Obliczenia

$$F_{CPU} = 1\,000\,000\text{ Hz}$$

$$T = 200\text{ ms} = 0,2\text{ s}$$

$$N = 1024\text{ (wartość preskalera)}$$

$$t_{stanWysoki} = 0,75 \cdot 0,2\text{ s} = 0,15\text{ s}$$

$$t_{stanNiski} = 0,25 \cdot 0,2\text{ s} = 0,05\text{ s}$$

$$OCR0 = \frac{F_{CPU} \cdot t}{N} - 1$$

Dla stanu wysokiego

$$OCR0 = \frac{1\,000\,000\text{ Hz} \cdot 0,15\text{ s}}{1024} - 1$$

$$OCR0 = \frac{150\,000}{1024} - 1$$

$$OCR0 = 146,48 - 1 \approx 145$$

Dla stanu niskiego

$$OCR0 = \frac{1\,000\,000\text{ Hz} \cdot 0,05\text{ s}}{1024} - 1$$

$$OCR0 = \frac{50\,000}{1024} - 1$$

$$OCR0 = 48,82 - 1 \approx 47\text{ (zaokrąglane w dół)}$$

3. Opis kodu

Program realizuje funkcję asymetrycznego generatora sygnału prostokątnego, wykorzystując dynamiczną zmianę parametrów trybu CTC. Kluczowe elementy kodu to:

- **Inicjalizacja generatora:**

```
TCCR0 |= (1<<WGM01) | (1<<CS02) | (1<<CS00); // Ustawienie na: tryb CTC, preskaler 1024
OCR0 = 145; // Początkowa wartość dla stanu wysokiego (150 ms)
```

- **Pętla synchronizacji i kontroli:**

```
while(!(TIFR&(1<<OCF0))); // Oczekiwanie na zakończenie aktualnego stanu
TIFR |= (1<<OCF0); // Zerowanie flagi dopasowania
PORTA ^= 0x01; // Zmiana stanu wyjścia na przeciwny
```

Program wstrzymuje działanie do momentu, aż licznik osiągnie wartość zapisaną w OCR0, po czym neguje stan najmłodszego bitu Portu A.

- **Dynamiczna zmiana wypełnienia:**

```
if(licznik == 1){
    OCR0 = 47; // Załadowanie czasu trwania stanu niskiego (50 ms)
}
if(licznik == 2){
    OCR0 = 145; // Załadowanie czasu trwania stanu wysokiego (150 ms)
    licznik = 0;
}
licznik++;
```

Wykorzystanie zmiennej pomocniczej licznik pozwala na naprzemienne wpisywanie różnych wartości do rejestru OCR0. Dzięki temu każdy kolejny cykl pracy timera ma inną długość, co skutkuje uzyskaniem sygnału o wypełnieniu 75% (150 ms stanu wysokiego i 50 ms stanu niskiego).

4. Kod

```
/*
 * 28_04_2026_KS.c
 *
 * Created: 28.04.2026 16:37:29
 * Author : student_pl
 */
#define F_CPU 1000000UL
#include <avr/io.h>
#include <util/delay.h>

int main(void)
{
    DDRA = 0xFF;
    PORTA = 0x00;
    TCCR0 |= (1<<WGM01) | (1<<CS02) | (1<<CS00);
    OCR0 = 145;
    int licznik = 1;
    while (1){
        while(!(TIFR&(1<<OCF0)));
        if(licznik == 1){
            OCR0 = 47;
        }
        if(licznik == 2){
            OCR0 = 145;
            licznik = 0;
        }
        licznik++;
        TIFR |= (1<<OCF0);
        PORTA ^= 0x01;
    }
    return 0;
}
```

Podsumowanie

Realizacja zadań laboratoryjnych pozwoliła na praktyczne poznanie mechanizmów odmierzenia czasu i zliczania zdarzeń w mikrokontrolerze ATmega32 przy użyciu Timera 0. Na podstawie wykonanych ćwiczeń można wyciągnąć następujące wnioski:

- **Przewaga trybu CTC nad pętlami programowymi:** Zastosowanie trybu CTC w Poleceniu 1 i 3 pozwala na precyzyjne odmierzenie interwałów czasowych bez obciążania procesora ciągłym liczeniem pustych instrukcji. Dzięki sprzętowemu zerowaniu licznika po osiągnięciu wartości w rejestrze OCR0, błędy kumulacyjne są zminimalizowane.
- **Rola rezystorów pull-up (ustawienia „1” na wejście pinu):** Aktywacja wewnętrznego podciągania pinu wejściowego do zasilania jest niezbędna do eliminacji stanów nieustalonych. Gwarantuje to stabilną pracę układu i zapobiega błędnemu zliczaniu przypadkowych zakłóceń elektromagnetycznych jako naciśnięć przycisku.
- **Eliminacja drgań styków:** Zastosowanie programowego opóźnienia `_delay_ms(200)` w procesie odczytu stanu licznika pozwoliło na stabilizację wyniku i uniknięcie zjawiska wielokrotnego zliczania jednego naciśnięcia przycisku.
- **Elastyczność generatorów sygnału:** Polecenie 3 pokazało, że poprzez dynamiczną zmianę wartości rejestru OCR0 w trakcie pracy programu, możliwe jest generowanie sygnałów PWM o niemal dowolnym okresie i wypełnieniu, co znajduje szerokie zastosowanie w sterowaniu np. jasnością diod LED czy prędkością silników.